

Relatório Técnico — Parte Teórica

Entrega contínua de uma plataforma de pedidos em microsserviços

Disciplina: Cloud DevOps Engineering | Caso: Loja Veloz — “Pedidos Veloz”

Do Docker Compose ao Kubernetes com observabilidade e CI/CD

Aluno: Bruno Maximino dos Reis

1. Arquitetura de microsserviços e o papel do DevOps em cloud-native

A arquitetura de microsserviços decompõe uma aplicação em serviços pequenos, independentes e organizados em torno de capacidades de negócio. Cada serviço tem seu próprio ciclo de vida, pode ser desenvolvido, implantado e escalado de forma autônoma e se comunica com os demais por interfaces bem definidas (HTTP/REST, gRPC ou mensageria assíncrona). O estilo se opõe ao monólito, em que todas as funcionalidades compartilham o mesmo processo e a mesma cadência de deploy — exatamente a dor relatada pela Loja Veloz, onde cada time sobe partes do sistema “como dá” e qualquer alteração coloca em risco todo o ambiente.

Os benefícios — autonomia de times, escalabilidade seletiva, isolamento de falhas e liberdade tecnológica — vêm acompanhados de custos: comunicação em rede, consistência distribuída e, sobretudo, complexidade operacional. É aí que entra o DevOps. Mais do que ferramentas, DevOps é uma cultura que aproxima desenvolvimento e operações por meio de automação, responsabilidade compartilhada e feedback rápido. Em um ambiente **cloud-native** — aplicações projetadas para nuvem, empacotadas em contêineres, orquestradas dinamicamente e operadas de forma elástica — o DevOps materializa-se em pipelines de entrega contínua, infraestrutura como código e telemetria abrangente. A metodologia **12-Factor App** formaliza esses princípios: configuração por ambiente, processos stateless, logs como streams, descartabilidade e paridade entre dev e produção.

2. Containerização: Docker e Kubernetes — quando usar cada um

Containerização é o empacotamento de uma aplicação com todas as suas dependências em uma unidade isolada e portátil, que executa de forma idêntica em qualquer ambiente. Contêineres compartilham o kernel do sistema operacional hospedeiro, o que os torna muito mais leves e rápidos que máquinas virtuais, sem o overhead de um SO completo por instância. Isso resolve o clássico “na minha máquina funciona”, garantindo paridade entre desenvolvimento e produção.

Docker e Kubernetes operam em camadas diferentes e são complementares, não concorrentes. O Docker é a plataforma de *construção e execução* de contêineres: a partir de um Dockerfile, gera-se uma imagem versionada e imutável. O Kubernetes é o *orquestrador*: gerencia o ciclo de vida de muitos contêineres em um cluster de máquinas, cuidando de agendamento, escala, autocorreção, rede e descoberta de serviços.

Aspecto	Docker (+ Compose)	Kubernetes
Escopo	Build e execução de contêineres em um host	Orquestração em cluster de múltiplos nós
Quando usar	Desenvolvimento local, build de imagens, stacks simples	Produção, alta disponibilidade, escala automática
Recuperação de falha	Manual / reinício básico	Autocorreção, reagendamento, probes
Escala	Limitada ao host	Horizontal automática (HPA) entre nós

Na prática: usa-se **Docker Compose** para padronizar o ambiente local — subir toda a stack com um comando — e **Kubernetes** para produção, onde resiliência, escala sob demanda e governança são

obrigatórias. Foi exatamente a divisão adotada neste projeto.

3. Fundamentação teórica

3.1 Orquestração de contêineres

Orquestração é a automação do agendamento, da implantação, da rede, do balanceamento de carga e do dimensionamento de contêineres em um conjunto de máquinas. No Kubernetes, o estado desejado é declarado (quantas réplicas, qual imagem, quais limites de recurso) e o *control plane* trabalha continuamente para reconciliar o estado real com o declarado. Os objetos centrais são o **Pod** (menor unidade implantável), o **Deployment** (gerencia réplicas e atualizações controladas), o **Service** (descoberta e balanceamento interno) e os **ConfigMaps/Secrets** (configuração e credenciais desacopladas da imagem). Probes de *liveness* e *readiness* permitem ao cluster reiniciar processos travados e rotear tráfego apenas para instâncias prontas.

3.2 CI/CD em ambientes distribuídos

Integração Contínua (CI) é a prática de integrar e validar mudanças com frequência: a cada commit, o pipeline executa lint, testes automatizados e a construção das imagens, oferecendo feedback rápido. **Entrega/Implantação Contínua (CD)** automatiza a publicação dos artefatos versionados em um registry e seu deploy nos ambientes, com *gates* de qualidade e segurança entre as etapas. Em sistemas distribuídos isso é vital: com muitos serviços evoluindo em paralelo, só a automação garante que cada um seja testado, escaneado e versionado de forma consistente. Credenciais sensíveis nunca ficam no código — são injetadas como *secrets* do pipeline. Estratégias de release (rolling, blue-green, canary) reduzem o risco de cada implantação.

3.3 Observabilidade: métricas, logs e traces

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir de seus sinais externos — essencial quando uma única requisição atravessa vários serviços. Apoia-se em três pilares:

- **Métricas** — séries temporais numéricas e agregáveis (latência, throughput, taxa de erro, uso de CPU). Respondem “o que” está acontecendo e alimentam alertas e autoescala. Padrão de mercado: Prometheus.
- **Logs** — registros discretos de eventos. Tratados como streams estruturados (JSON) e centralizados, ajudam a entender “por que” algo ocorreu.
- **Traces (rastreamento distribuído)** — seguem uma requisição ponta a ponta, correlacionando os serviços por um *trace_id* propagado. Revelam “onde” está o gargalo ou a falha. Padrão aberto: OpenTelemetry, com backends como Jaeger/Tempo.

4. Caso concreto de referência: Online Boutique (Google)

Como referência pública e madura, adotamos o **Online Boutique** (GoogleCloudPlatform/microservices-demo), aplicação de e-commerce cloud-native mantida pelo Google. Ela é composta por cerca de uma dezena de microsserviços escritos em múltiplas linguagens, que se comunicam predominantemente via gRPC e roda em qualquer cluster Kubernetes — de um minikube local ao GKE gerenciado. O projeto demonstra na prática os mesmos padrões exigidos por este desafio: orquestração no Kubernetes, separação por serviço, observabilidade (tracing e métricas), service mesh (Istio) e deploy declarativo. Ele valida as escolhas da nossa proposta e serve de espelho arquitetural para a “Pedidos Veloz”, que adota uma versão enxuta e didática dos mesmos princípios (HTTP/REST no lugar de gRPC, quatro serviços de domínio em vez de uma dezena), suficiente para o MVP de quatro semanas.

5. Justificativa das principais decisões arquiteturais

- **Microsserviços por domínio** (Gateway, Pedidos, Pagamentos, Estoque): isolam falhas e permitem escalar só o que sofre pico — o serviço de Pedidos, no caminho crítico da campanha promocional.
- **Docker Compose no local, Kubernetes em produção**: paridade dev/prod (12-Factor) com baixo custo de entrada e resiliência onde importa.

- **Imagens multi-stage e usuário não-root:** imagens enxutas, superfície de ataque reduzida e versionamento por SHA/semver.
- **CI/CD com testes e scan de vulnerabilidades:** cada mudança é validada antes de ir a produção, reduzindo o risco de deploy.
- **Mensageria para o evento “PedidoCriado”:** desacopla efeitos colaterais (notificações, antifraude) do fluxo síncrono, melhorando resiliência e tempo de resposta.
- **Observabilidade desde o MVP:** métricas, logs estruturados e tracing aumentam a rastreabilidade de falhas entre serviços, a maior dor relatada.

Referências (fontes primárias)

Kubernetes — Documentation (Pods, Deployments, Services, ConfigMaps/Secrets, HPA, Pod Security).
Docker — Documentation (Dockerfile, multi-stage, Compose). The Twelve-Factor App (12factor.net).
HashiCorp Terraform — Documentation. GitHub Actions — Documentation. OpenTelemetry e Istio — Documentation. GoogleCloudPlatform/microservices-demo (Online Boutique), GitHub. “, ”meta